

WP Agentic AI

Praktikum – Übungsblätter

Prof. Dr. Thomas Clemen | HAW Hamburg | SS 2026

Terminübersicht

Termin	Thema	Vorlesungskapitel
01	Projektdefinition & Agentenanalyse	Kap. 01: Agents (Fundamentals)
02	Architekturentwurf: Kognitive Schichten	Kap. 02: Perception, Memory & Planning
03	Erste Implementierung: LLM, RAG & Tool Use	Kap. 03: LLMs, RAG & Tool Use
04	Architektur skalieren: Multi-Agent	Kap. 04: Multi-Agent Systems
05	Engineering: Robustheit & Testing	Kap. 05: Agentic AI Engineering
06	Safety, Guardrails & Präsentation	Kap. 06: Safety + Guardrails

Übungsblatt 01

Projektdefinition & Agentenanalyse

Begleitend zu Kapitel 01: Org + Agents (Fundamentals)

Leitfrage

Was soll euer Agent tun – und warum ist das ein Agent?

In diesem ersten Praktikumstermin bildet ihr eure Projektgruppen, wählt ein Thema und analysiert euer geplantes System mit den Werkzeugen aus Kapitel 01. Am Ende habt ihr eine präzise Spezifikation dessen, was euer Agent ist, in welcher Umgebung er agiert und was ihn von einer gewöhnlichen Software unterscheidet.

Meilenstein 1: Teambildung & Themenwahl

- Bildet Gruppen à max. 5 Personen.
- Einigt euch auf ein Projektthema. Euer Agent muss mindestens folgende Eigenschaften haben: (a) Er nimmt seine Umgebung wahr (Perception), (b) er trifft Entscheidungen (Planning) und (c) er führt Aktionen aus (Action).
- Formuliert in 2–3 Sätzen, was euer Agent können soll (Elevator Pitch).

Meilenstein 2: PEAS-Analyse

- Erstellt eine vollständige PEAS-Tabelle für euer System: Performance Measure (Woran erkennt man Erfolg?), Environment (In welcher Umgebung agiert der Agent?), Actuators (Welche Aktionen kann er ausführen?), Sensors (Welche Eingaben erhält er?).
- Ordnet die Umgebungseigenschaften eures Systems auf den sieben Dimensionen ein (fully/partially observable, deterministic/stochastic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent, known/unknown). Begründet jede Einordnung.
- Bestimmt, welchem Agententyp aus der Taxonomie euer System am nächsten kommt (Simple Reflex → Learning Agent) und begründet eure Wahl.

Meilenstein 3: Ergebnissicherung

- Erstellt ein kurzes Dokument (1–2 Seiten) mit Elevator Pitch, PEAS-Tabelle, Umgebungseinordnung und Agententyp.
- Speichert euer Ergebnis im gemeinsamen Repository (Git).

Abgabe / Deliverable

Projektdokument mit Elevator Pitch, PEAS-Analyse, Umgebungseinordnung und Agententyp.

Tipp: Denkt an die Drei-Fenster-Demo aus der Vorlesung: Euer Agent sollte mehr können als ein reiner Chatbot – aber ihr müsst auch nicht gleich ein autonomes Fahrzeug bauen. Findet die richtige Komplexitätsstufe!

Übungsblatt 02

Architekturentwurf: Die kognitiven Schichten

Begleitend zu Kapitel 02: Perception, Memory & Planning

Leitfrage

Wie nimmt euer Agent die Welt wahr, was merkt er sich, und wie entscheidet er?

Kapitel 02 hat die kognitive 4-Schichten-Architektur mit konkreten Mechanismen gefüllt. Heute übertragt ihr dieses Analyseraster auf euer eigenes Projekt und entwerft die Architektur eures Agenten – noch ohne Implementierung, aber mit klaren Designentscheidungen.

Meilenstein 1: Schichten-Mapping

- Füllt für euer Projekt die kognitive 4-Schichten-Architektur aus: Perception – Welche Inputs erhält euer Agent? (Text, APIs, Sensordaten, Dateien?) Wie werden sie vorverarbeitet?
- Memory – Welche Memory-Typen braucht euer Agent? (Working Memory / Kurzzeit, Episodic Memory / Was ist passiert?, Semantic Memory / Weltwissen). Wo reicht das Context Window, wo braucht ihr mehr?
- Planning – Ist euer Agent eher reaktiv oder deliberativ? Braucht er hierarchische Pläne oder reicht eine einfache Wenn-Dann-Logik?
- Action – Welche Aktionen soll euer Agent ausführen? Welche davon haben Seiteneffekte in der realen Welt?

Meilenstein 2: Architekturdiagramm

- Zeichnet ein Architekturdiagramm eures Agenten, das die vier Schichten und den Datenfluss zwischen ihnen zeigt.
- Markiert im Diagramm, welche Schichten ihr als kritisch einschätzt (= wo die größten Herausforderungen liegen).
- Identifiziert mindestens eine Schwäche in eurem Entwurf: Welche Schicht ist am schwächsten besetzt? Was fehlt noch?

Meilenstein 3: Technologie-Vorauswahl

- Recherchiert kurz, welche technischen Bausteine ihr für die einzelnen Schichten einsetzen könntet (z. B. welches LLM, welche Datenquellen, welche APIs).
- Haltet offene Fragen fest, die sich im nächsten Termin (LLMs + RAG + Tool Use) klären werden.
- Aktualisiert euer Projektdokument um den Architekturentwurf.

Abgabe / Deliverable

Architekturdiagramm mit 4-Schichten-Mapping, Technologie-Vorauswahl und identifizierten Schwächen.

Tipp: Das Pizza-Szenario aus der Vorlesung ist ein gutes Testinstrument: Spielt euren Use Case einmal mental durch und fragt euch bei jedem Schritt, welche Schicht gerade aktiv ist. Wenn eine Schicht nie vorkommt, stimmt entweder der Use Case oder der Entwurf nicht.

Übungsblatt 03

Erste Implementierung: LLM, RAG & Tool Use

Begleitend zu Kapitel 03: LLMs, RAG & Tool Use

Leitfrage

Wie macht ihr euer LLM zum handlungsfähigen Agenten?

Heute wird es konkret: Ihr bindet ein LLM an, gebt eurem Agenten Gedächtnis (RAG) und Hände (Tool Use). Am Ende dieses Termins soll euer Agent zum ersten Mal etwas Sinnvolles tun – nicht nur Text produzieren.

Meilenstein 1: LLM-Anbindung & Grundgerüst

- Richtet die Entwicklungsumgebung ein (Python, API-Keys, Dependencies).
- Implementiert einen ersten LLM-Call für euren Use Case mit einem passenden System Prompt.
- Testet: Beantwortet das LLM eure domänenspezifischen Fragen korrekt? Wo zeigen sich die Grenzen (fehlende Aktualität, fehlendes Spezialwissen, keine Aktionen)?

Meilenstein 2: RAG oder Tool Use – die erste Lücke schließen

- Entscheidet: Welche Lücke schließt ihr zuerst – Memory (RAG) oder Action (Tool Use)?
- Option A – RAG: Bereitet eure Wissensbasis vor (Texte, PDFs, Daten), implementiert eine einfache Retrieval-Pipeline (z. B. mit einem Vector Store) und testet, ob der Agent jetzt bessere Antworten gibt.
- Option B – Tool Use: Definiert 2–3 Tools/Functions, die euer Agent aufrufen können soll. Implementiert sie zunächst als Mock-Funktionen und bindet sie per Function Calling an das LLM an.
- Dokumentiert das Ergebnis: Was kann euer Agent jetzt, was er vorher nicht konnte?

Meilenstein 3: Erster End-to-End-Durchlauf

- Führt einen vollständigen Durchlauf eures Agenten durch: Eingabe → Verarbeitung → Ausgabe/Aktion.
- Haltet fest: Was funktioniert? Was bricht? Welche Lücke (RAG oder Tool Use) müsst ihr beim nächsten Mal noch schließen?

Abgabe / Deliverable

Lauffähiger Prototyp mit LLM-Anbindung und mindestens einem realisierten Baustein (RAG oder Tool Use). Kurze Doku des End-to-End-Durchlaufs.

Tipp: Fangt mit Mock-Tools an! Eine Funktion, die einfach hartcodierte Daten zurückgibt, reicht völlig aus, um den Agent-Loop zu testen. Die echte API-Anbindung kann später kommen.

Übungsblatt 04

Architektur skalieren: Multi-Agent oder Vertiefung

Begleitend zu Kapitel 04: Multi-Agent Systems

Leitfrage

Braucht euer System mehrere Agenten – und wenn ja, wie koordinieren sie sich?

Kapitel 04 zeigt, wann und warum ein einzelner Agent an seine Grenzen stößt. Heute entscheidet ihr: Profitiert euer Projekt von einer Multi-Agent-Architektur, oder vertieft ihr die Single-Agent-Implementierung? Beide Wege sind legitim – die Begründung zählt.

Meilenstein 1: Architekturentscheidung

- Prüft anhand der Heuristik aus der Vorlesung: Gibt es in eurem System unabhängige Teilaufgaben, die von Spezialisierung profitieren? Würde Parallelisierung den Durchsatz erhöhen?
- Entscheidet begründet: (A) Multi-Agent – Ihr führt einen Orchestrator und mindestens zwei spezialisierte Worker ein, oder (B) Vertiefung – Euer System bleibt single-agent, und ihr schließt die zweite Lücke aus Termin 3 (RAG oder Tool Use) und erweitert die Funktionalität.
- Dokumentiert eure Entscheidung und die Begründung.

Meilenstein 2: Implementierung

- Pfad A (Multi-Agent): Definiert die Rollen eurer Agenten (System Prompt + Toolset + Scope pro Agent). Implementiert die Kommunikation zwischen Orchestrator und Workern (Shared State oder Message Passing). Führt einen Durchlauf durch, bei dem mindestens zwei Worker zusammenarbeiten.
- Pfad B (Vertiefung): Schließt die zweite Lücke (RAG oder Tool Use). Implementiert ein ReAct-Pattern (Thought → Action → Observation) für komplexere Aufgaben. Erweitert die Funktionalität um mindestens ein neues Feature.

Meilenstein 3: Zwischenpräsentation

- Zeigt euren aktuellen Stand kurz den anderen Gruppen (max. 3 Minuten pro Gruppe).
- Holt euch Feedback: Was ist unklar? Was fehlt? Wo seht ihr Risiken?
- Notiert das Feedback und priorisiert, was ihr in den letzten zwei Terminen angehen wollt.

Abgabe / Deliverable

Begründete Architekturentscheidung (Multi-Agent vs. Vertiefung). Erweiterte Implementierung. Feedback-Notizen.

Tipp: Multi-Agent ist kein Selbstzweck! Wenn euer Use Case mit einem Agent gut funktioniert, ist die bewusste Entscheidung dagegen genauso wertvoll wie die Entscheidung dafür. Entscheidend ist die Begründung.

Übungsblatt 05

Engineering: Robustheit, Testing & Observability

Begleitend zu Kapitel 05: Agentic AI Engineering

Leitfrage

Wie wisst ihr, ob euer Agent zuverlässig funktioniert – und wie findet ihr Fehler?

Euer Agent läuft – aber ist er zuverlässig? Heute geht es um die Ingenieursarbeit: Fehlerbehandlung, Testing und Nachvollziehbarkeit. Diese Aspekte unterscheiden einen Prototypen von einem belastbaren System.

Meilenstein 1: Fehlerbehandlung

- Identifiziert mindestens drei Failure Modes eures Systems (z. B. API-Timeout, ungültige LLM-Antwort, leere Suchergebnisse, Parse-Fehler bei Structured Output).
- Implementiert für mindestens zwei davon eine Fehlerbehandlung (Retry, Fallback, Graceful Degradation).
- Testet: Provoziert den Fehler absichtlich und zeigt, dass euer System sinnvoll reagiert.

Meilenstein 2: Testing

- Schreibt mindestens drei Tests für euer System. Da LLM-basierte Systeme nicht-deterministisch sind, verwendet semantische Assertions statt exakter Stringvergleiche (z. B. Enthält die Antwort ein Datum? Ist die Empfehlung konsistent mit den Eingabedaten? Ruft der Agent die richtigen Tools auf?).
- Führt die Tests aus und dokumentiert die Ergebnisse.
- Reflektiert: Welche Aspekte eures Systems sind schwer testbar? Warum?

Meilenstein 3: Observability & Tracing

- Implementiert ein einfaches Logging/Tracing, das für jeden Durchlauf nachvollziehbar macht: welche Eingabe, welche LLM-Calls, welche Tool-Aufrufe, welche Zwischenergebnisse, welches Endergebnis.
- Führt zwei Durchläufe mit derselben Eingabe durch und vergleicht die Traces. Wo unterscheiden sie sich? Ist das problematisch?
- Aktualisiert eure Dokumentation: Architekturdiagramm mit Fehlerbehandlung und Tracing-Punkten.

Abgabe / Deliverable

Robusterer Agent mit Fehlerbehandlung, mindestens drei semantischen Tests und einem Tracing-Mechanismus.

Tipp: Ein System ohne Tracing ist wie Autofahren ohne Armaturenbrett. Ihr wisst nicht, wie schnell ihr fahrt, wie viel Sprit noch da ist – und wenn etwas schiefgeht, wisst ihr nicht, warum.

Übungsblatt 06

Safety, Guardrails & Abschlusspräsentation

Begleitend zu Kapitel 06: Safety + Guardrails

Leitfrage

Was darf euer Agent nicht tun – und wie stellt ihr das sicher?

Im letzten Termin bringt ihr euer Projekt zum Abschluss. Zuerst implementiert ihr Guardrails, die verhindern, dass euer Agent unerwünschte Aktionen ausführt. Dann präsentiert jede Gruppe ihr fertiges System.

Meilenstein 1: Safety-Analyse & Guardrails

- Führt eine einfache Risikoanalyse für euer System durch: Was könnte schiefgehen? Welche unerwünschten Aktionen sind denkbar? Denkt dabei an die Angriffsvektoren aus der Vorlesung (Prompt Injection, Goal Misgeneralization, Unintended Side Effects).
- Implementiert mindestens zwei Guardrail-Mechanismen, z. B. Input-Filterung (bestimmte Anfragen ablehnen), Output-Monitoring (Antworten vor Auslieferung prüfen), Scope-Einschränkungen (Tools nur in definiertem Rahmen nutzbar), Human-in-the-Loop (bei kritischen Aktionen Bestätigung einholen).
- Demonstriert: Zeigt einen konkreten Fall, in dem euer Guardrail greift (z. B. eine bösartige Eingabe, die abgefangen wird).

Meilenstein 2: Präsentationsvorbereitung

- Bereitet eine Präsentation vor (max. 10 Minuten pro Gruppe), die folgende Punkte abdeckt: (1) Was tut euer Agent? (Elevator Pitch + Live-Demo), (2) Architektur: Welche kognitiven Schichten habt ihr wie realisiert?, (3) Technische Highlights: Was war die größte Herausforderung?, (4) Safety: Welche Guardrails habt ihr implementiert?, (5) Lessons Learned: Was würdet ihr beim nächsten Mal anders machen?

Meilenstein 3: Abschlusspräsentationen am 7.7.2026 in der Vorlesung

- Jede Gruppe präsentiert ihr Projekt (max. 10 min Präsentation + 5 min Fragen).
- Bewertet die anderen Projekte anhand der PEAS-Kriterien aus Termin 1: Ist das wirklich ein Agent? Welche Schichten sind stark/schwach besetzt?

Abgabe / Deliverable

Fertiges Projekt mit Guardrails, Projektdokumentation und Abschlusspräsentation.

Tipp: Die beste Präsentation ist eine Live-Demo, die funktioniert – und eine, die zeigt, was passiert, wenn jemand versucht, den Agent auszutricksen. Das ist deutlich spannender als Folien.